Deploying an Image Classification Application on AWS Using Docker, DVC, and CI/CD

MLOps Project Report

Vedant Singh Thakur
PRN: 23070126144
Symbiosis Institute of Technology, Pune
October 2026

GitHub: https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144
Application: http://15.252.12.60/

1. Introduction and Objectives

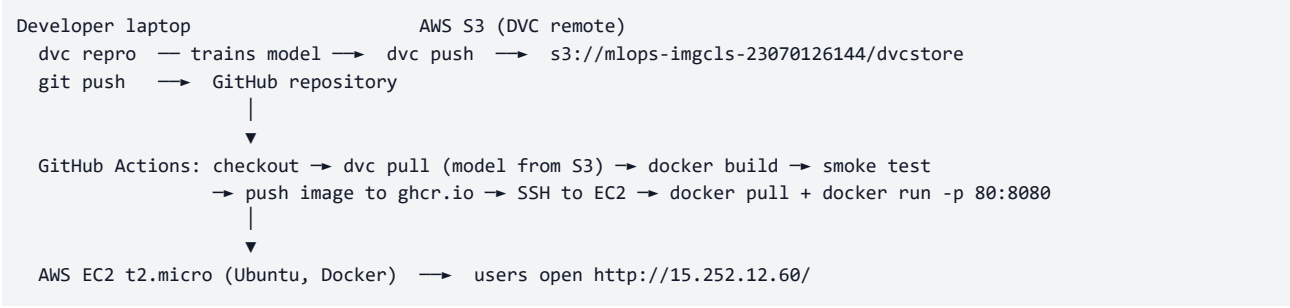
This project builds, versions and deploys an image classification application that recognises six types of natural and urban scenes (buildings, forest, glacier, mountain, sea, street) from the Intel Image Dataset. A MobileNetV2 transfer learning model is trained through a reproducible DVC pipeline, its artifacts are versioned in AWS S3, and a Flask web application serving the model is containerised with Docker and deployed automatically to an AWS EC2 instance by a GitHub Actions CI/CD workflow.

Objectives covered in this report:

- dvc.yaml defining the five-stage DVC pipeline (data collection, processing, feature extraction, training, evaluation).
- ci-cd.yaml implementing the GitHub Actions CI/CD workflow (.github/workflows/ci-cd.yaml).
- A Dockerfile that containerises the image classification web application.
- AWS S3 configured as the DVC remote, an AWS EC2 instance, and the Dockerised application deployed on it.
- Verification of the deployment through the public IP of the EC2 instance (http://15.252.12.60/).

2. Architecture and Tools

End-to-end workflow:



Component	Technology	Purpose
ML pipeline	Python 3, TensorFlow 2.21 / Keras 3, OpenCV, scikit-learn	Data processing, MobileNetV2 training and evaluation
Pipeline orchestration and versioning	DVC 3.67 (dvc.yaml, params.yaml, dvc.lock)	Reproducible stages, dependency tracking, metrics
Remote storage	AWS S3 (ap-south-1)	DVC remote holding the trained model, features and metrics
Web application	Flask + Gunicorn	Upload page, JSON prediction API and health check
Containerisation	Docker (python:3.11-slim, tensorflow-cpu)	Portable runtime image for the app
CI/CD	GitHub Actions + GitHub Container Registry	Build, test, publish and deploy on every push
Hosting	AWS EC2 t2.micro, Ubuntu, free tier	Runs the container on port 80 with a public IP

3. Project Structure

```
MLOps_ImageClassification_23070126144/
├── .github/workflows/ci-cd.yaml   GitHub Actions CI/CD workflow
├── .dvc/config                    DVC remote configuration (S3)
├── src/                           pipeline stages
│   ├── data_collection.py         downloads the Kaggle dataset
│   ├── data_processing.py         resizes images, 70/20/10 train/validation/test split
│   ├── feature_selection.py       MobileNetV2 feature extractor (ImageNet weights)
│   ├── model_training.py          classification head training
│   ├── model_evaluation.py        accuracy, precision, recall, F1, confusion matrix
│   ├── config.py, utils.py        shared helpers
├── app.py                         Flask web application
├── Dockerfile, .dockerignore      container definition
├── dvc.yaml, dvc.lock, params.yaml
├── requirements.txt               pipeline dependencies
├── requirements-app.txt           runtime dependencies of the container
├── samples/                       one test image per class (used by CI smoke test)
├── screenshots/                   screenshots used in this report
└── report/                        this report and its generator
```

4. Step 1 - DVC Pipeline (dvc.yaml)

All hyper-parameters (image size 224x224, batch size 32, 15 epochs, Adam with learning rate 1e-4, dropout 0.3, 256 dense units, split ratios, augmentation settings and seed) live in params.yaml. dvc.yaml declares every stage with its command, code and data dependencies, the parameters it reads and the outputs it produces, so DVC re-runs only the stages whose inputs changed.

Stage	Command	Main dependencies	Outputs
data_collection	python -m src.data_collection	params: dataset_url, dataset_path	data/raw
data_processing	python -m src.data_processing	data/raw, image_size, splits	data/processed, artifacts/processed
feature_selection	python -m src.feature_selection	data/processed, model_name	artifacts/features
model_training	python -m src.model_training	data/processed, artifacts/features, epochs, lr	artifacts/model
model_evaluation	python -m src.model_evaluation	data/processed, artifacts/model	metrics.json, confusion_matrix.png

Commands used:

```
git init -b main                     # DVC tracks pipeline state in Git
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
dvc repro                            # runs all five stages and writes dvc.lock
dvc dag                              # shows the stage graph
dvc metrics show                     # prints evaluation metrics
```

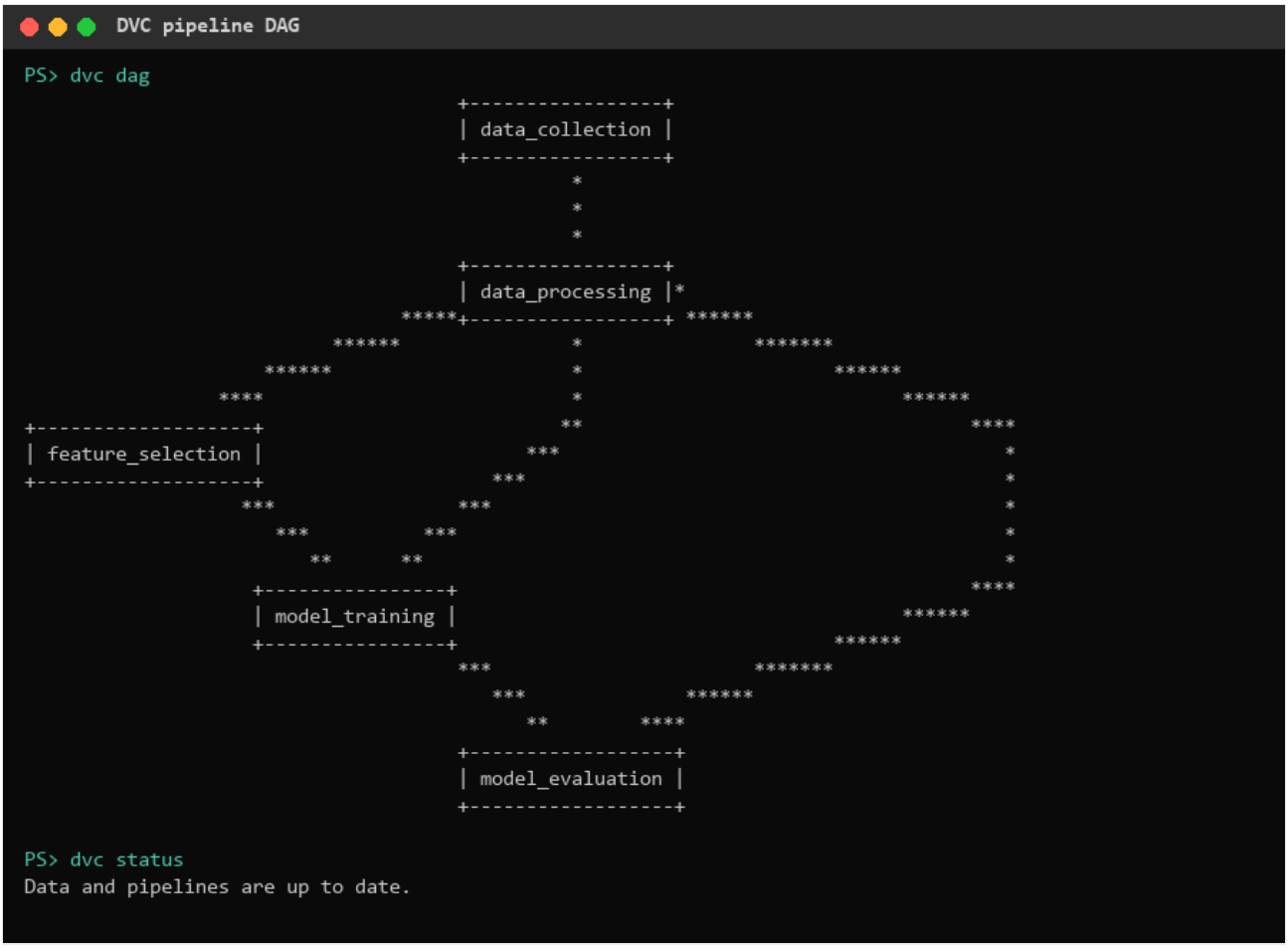


Figure 1: DVC pipeline DAG (dvc dag) and dvc status after the run

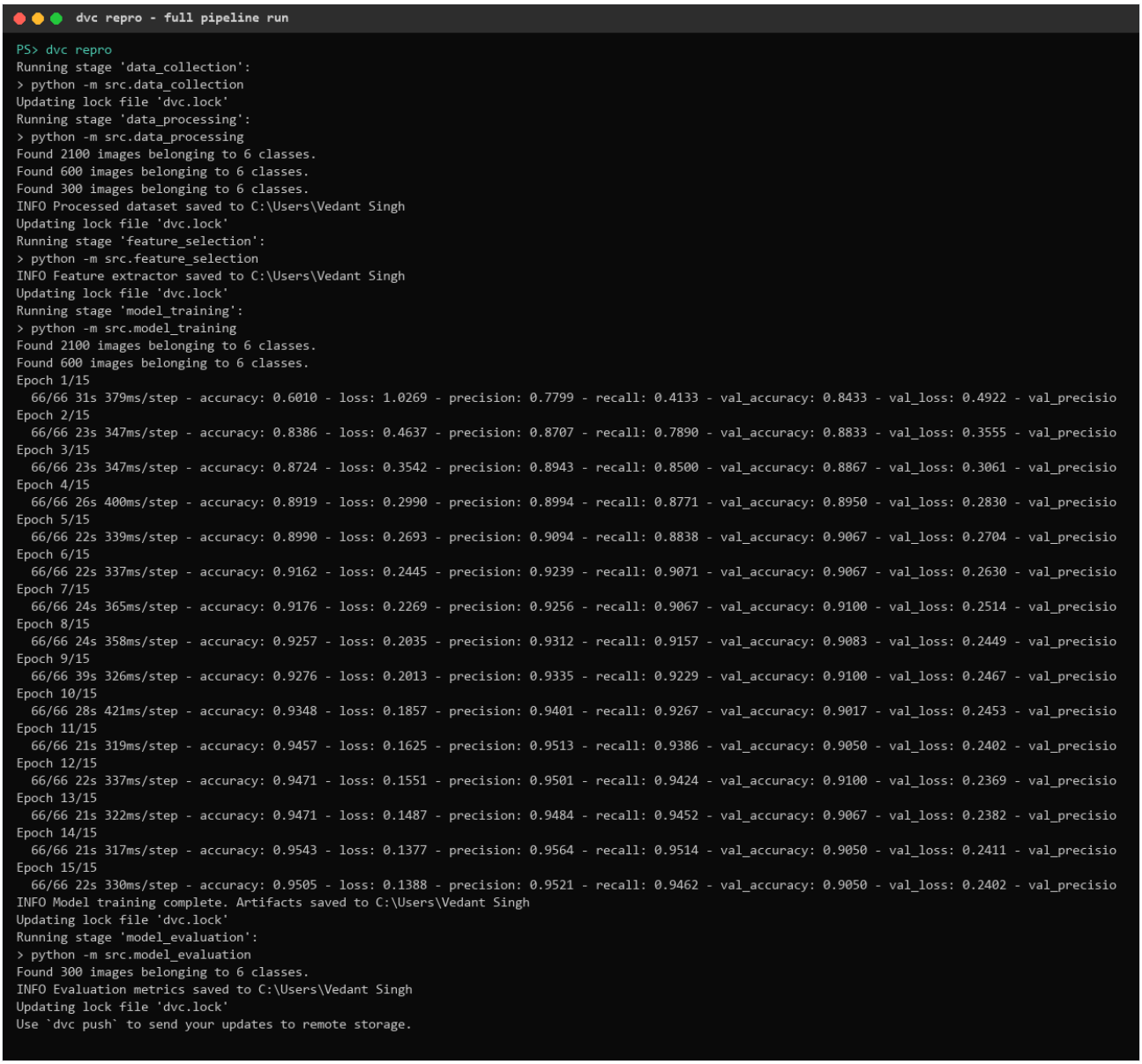


Figure 2: dvc repro executing all five stages, including 15 training epochs

The Intel dataset used (3,000 images, 6 classes) is split into 2,100 training, 600 validation and 300 test images. Frozen MobileNetV2 features with a 256-unit dense head reach about 91% validation accuracy within 5 epochs.

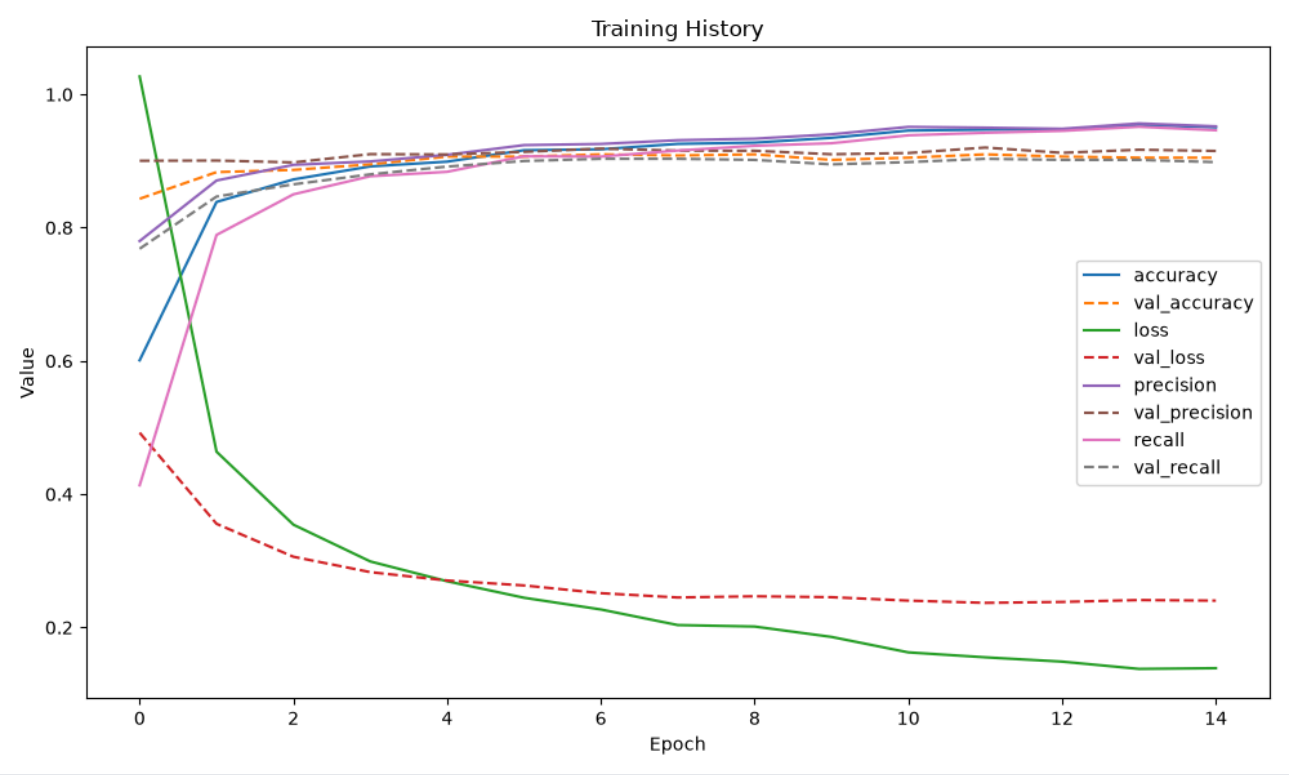


Figure 3: Training and validation curves produced by the model_training stage

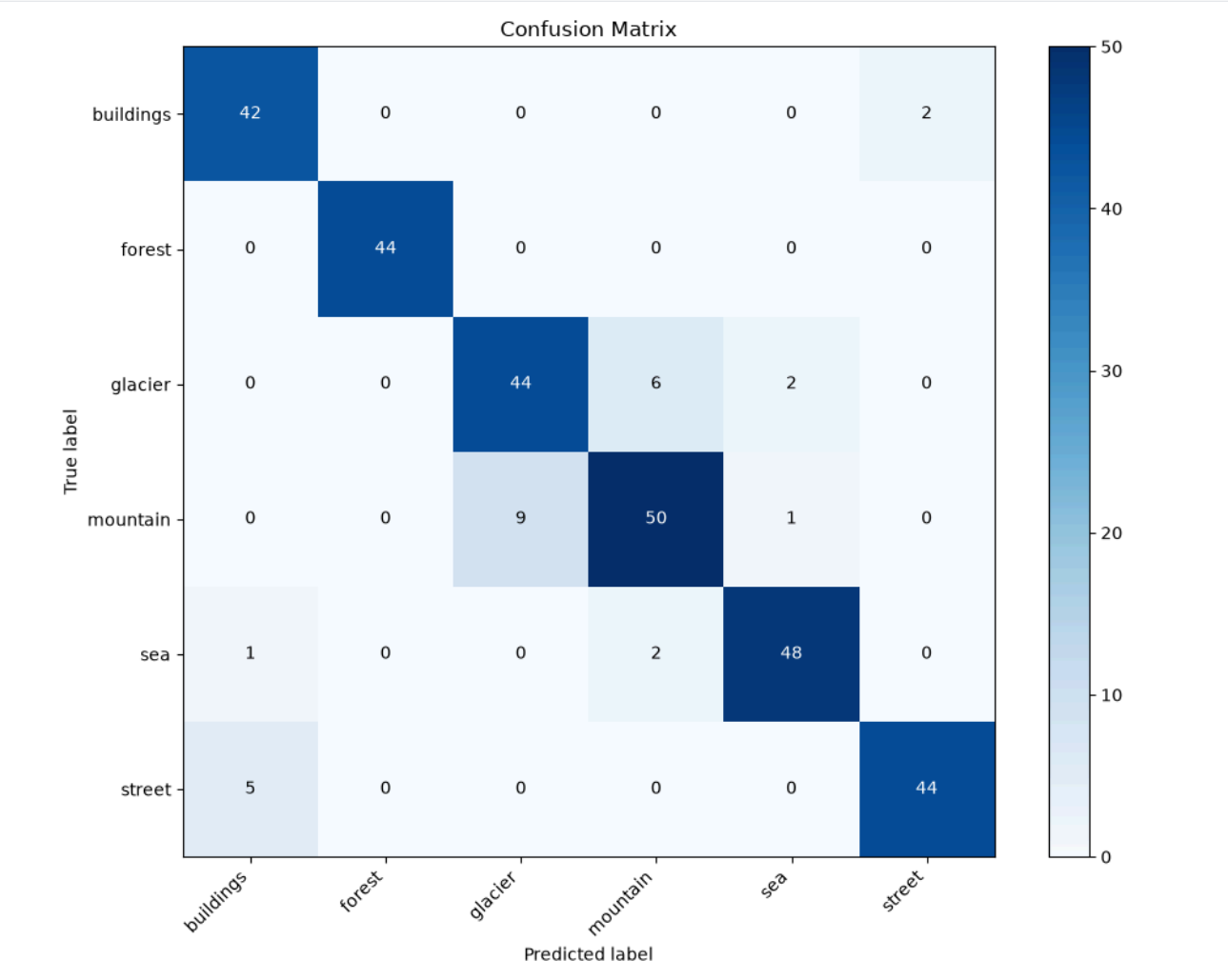


Figure 4: Confusion matrix on the 300 held-out test images (model_evaluation stage)

5. Step 2 - AWS S3 as the DVC Remote

1. Created IAM user mlops-dvc. with programmatic access only and the AmazonS3FullAccess policy, then created an access key and stored it locally with aws configure.
2. Created the S3 bucket mlops-imgcls-23070126144 in ap-south-1 (Mumbai) with Block Public Access enabled.
3. Configured the bucket as the default DVC remote and pushed the pipeline artifacts.

```
pip install "dvc[s3]"
dvc remote add -d storage s3://mlops-imgcls-23070126144/dvcstore
dvc remote modify storage region ap-south-1
dvc push feature_selection model_training model_evaluation
git add .dvc/config dvc.lock && git commit -m "Configure S3 as DVC remote storage"
```

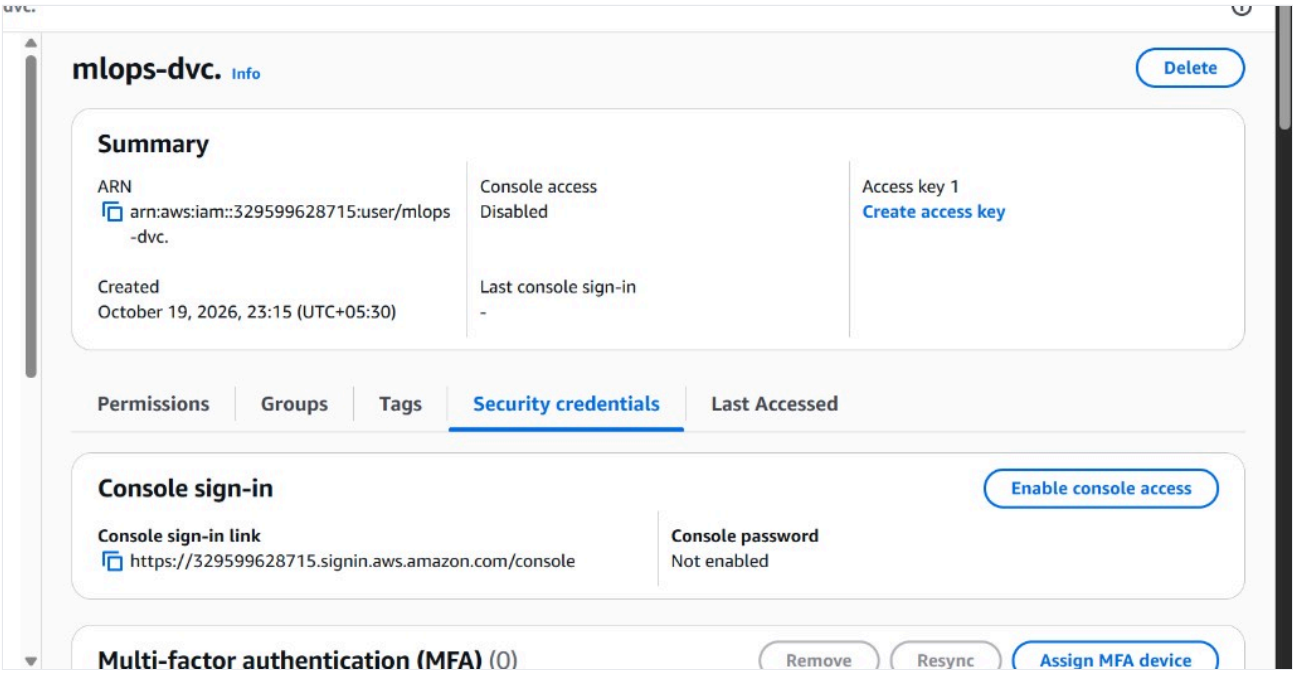


Figure 5: IAM user mlops-dvc. used by DVC and GitHub Actions

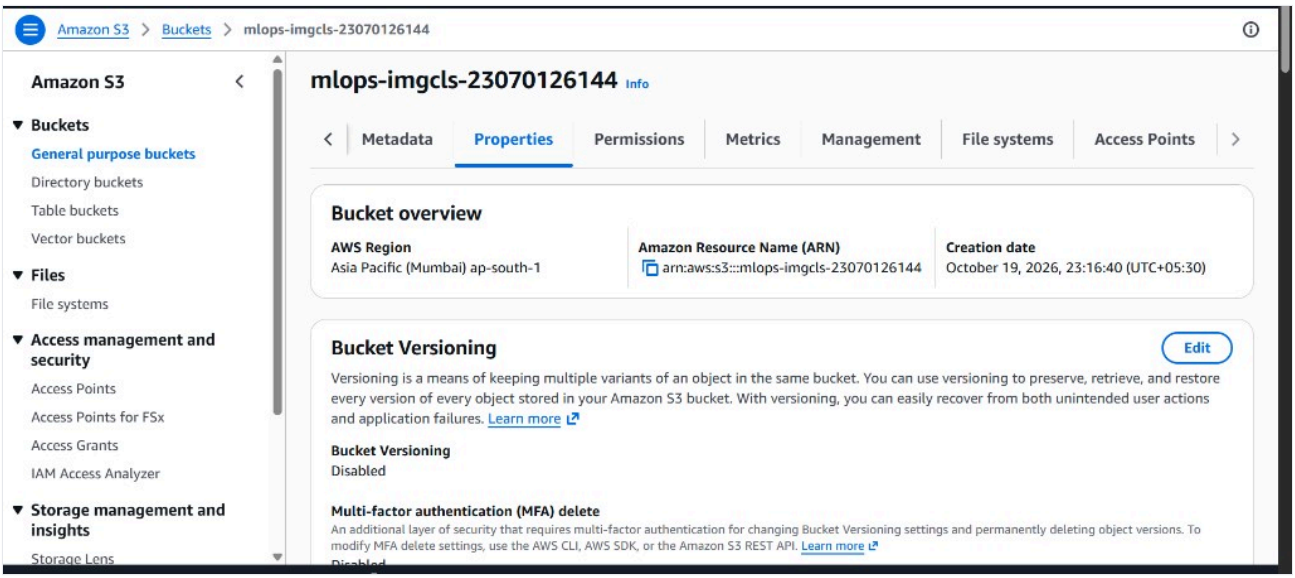


Figure 6: S3 bucket mlops-imgcls-23070126144 in ap-south-1

DVC metrics and push to S3 remote

```
PS> dvc metrics show
Path                          accuracy    f1_score    precision    recall
artifacts\metrics\metrics.json  0.90667     0.90664     0.90755      0.90667

PS> dvc remote list
storage s3://mlops-imgcls-23070126144/dvcstore  (default)

PS> dvc push feature_selection model_training model_evaluation
9 files pushed

PS> aws s3 ls s3://mlops-imgcls-23070126144/dvcstore/ --recursive --human-readable --summarize
2026-10-19 23:32:47   13.0 MiB dvcstore/files/md5/41/96130bc2647ecda4dd479cfa5351c6
2026-10-19 23:32:47   64.3 KiB dvcstore/files/md5/58/81f91b4e90bc3fb14f9d05e88e322a
2026-10-19 23:32:49  142 Bytes dvcstore/files/md5/60/4c9726fd6e6dde0cc122cc35b48d78
2026-10-19 23:32:47    3.1 KiB dvcstore/files/md5/67/b94cda673e1478bcb1866905a10285
2026-10-19 23:32:47   13.0 MiB dvcstore/files/md5/97/f06d97e9228629a0361cdc866159e2
2026-10-19 23:32:49  355 Bytes dvcstore/files/md5/b3/b1a9dd35dc35a321212df074cd7f18.dir
2026-10-19 23:32:49   37.6 KiB dvcstore/files/md5/ba/d0c1359fe81161f3457a67cdeafcb0
2026-10-19 23:32:47  112 Bytes dvcstore/files/md5/d3/e202c259cbc939b6e36ee1915d6e6e.dir
2026-10-19 23:32:45    9.2 MiB dvcstore/files/md5/ee/92a87f30ee9eb9dfa946a624313030

Total Objects: 9
   Total Size: 35.2 MiB
```

Figure 7: dvc metrics show, dvc push to the S3 remote and the resulting S3 objects

Only the model, feature extractor and metrics stages are pushed (9 objects, 35 MB). The raw and processed images are reproducible from Kaggle by the data_collection stage, so leaving them out keeps the bucket well within the free tier's 2,000 PUT requests per month. A fresh clone running dvc pull model_training model_evaluation restores the model exactly as recorded in dvc.lock, which is what the CI pipeline does.

6. Step 3 - Flask Web Application

app.py loads the trained model (artifacts/model/mobilenetv2_intel_classifier.keras) once at start-up, resizes uploaded images to 224x224 RGB, rescales them to [0, 1] exactly like the training generators, and returns the predicted class with per-class probabilities. Endpoints:

- GET / - HTML page to upload an image and view the prediction with probability bars.
- POST /predict - JSON API, e.g. curl -F file=@samples/forest.jpg http://15.252.12.60/predict
- GET /health - health check used by Docker HEALTHCHECK and the CI/CD pipeline.
- Invalid uploads return HTTP 400; uploads are limited to 10 MB.

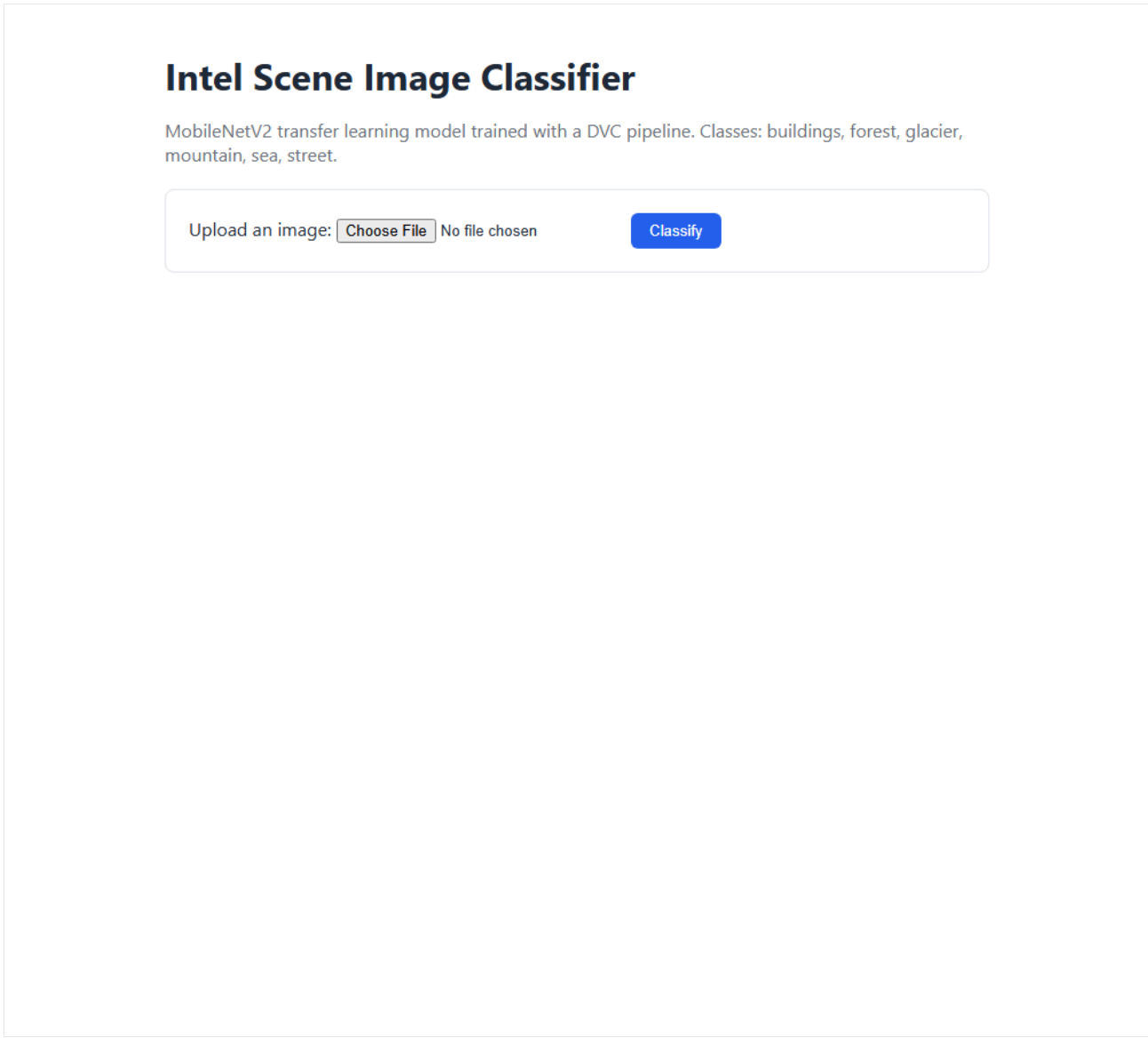


Figure 8: Application home page running locally in Docker

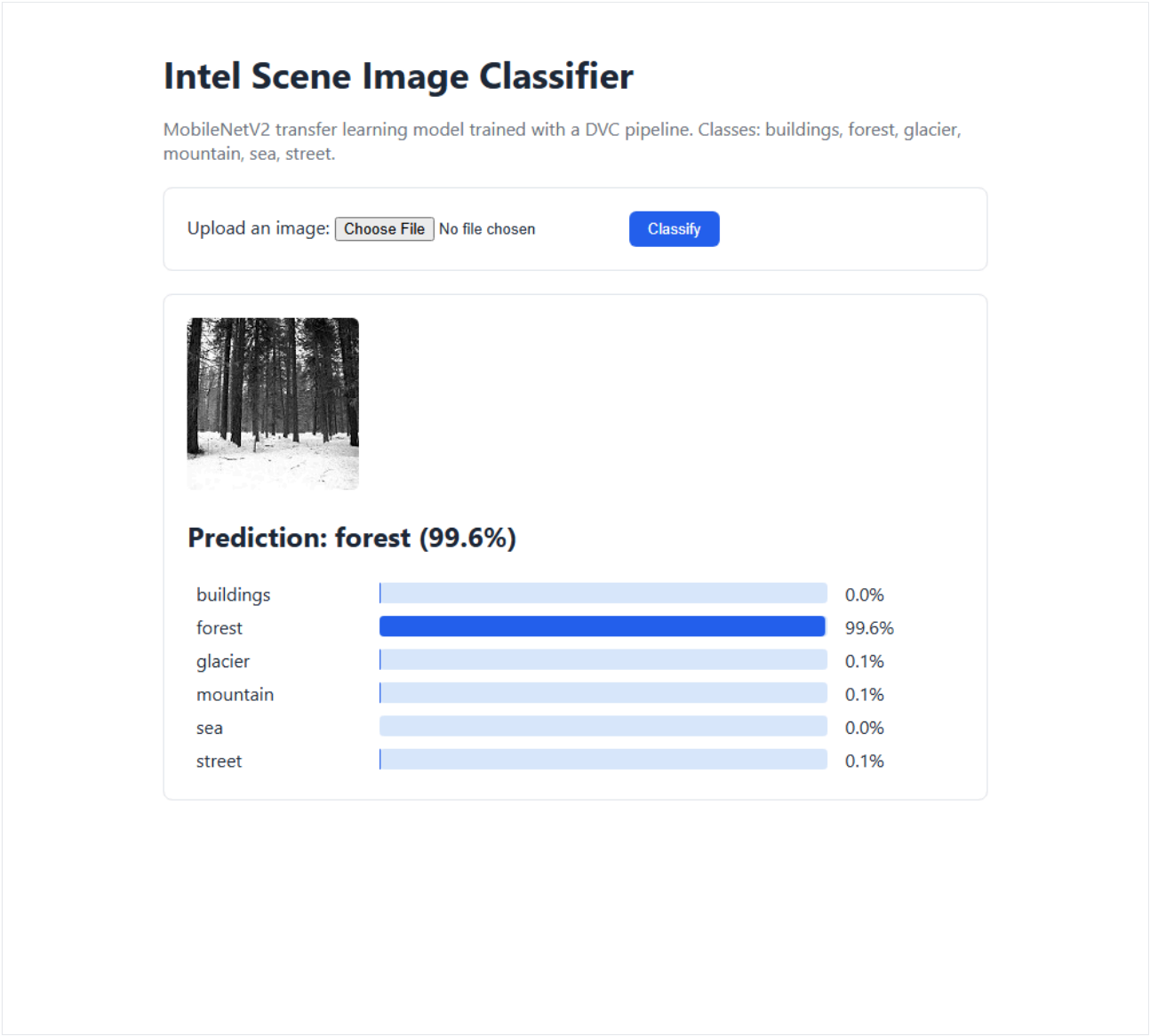


Figure 9: Local prediction for a forest test image

7. Step 4 - Dockerfile and Docker Build

The image is based on python:3.11-slim and installs only the runtime dependencies (tensorflow-cpu 2.21.0 and keras 3.15.1, the same versions used for training, plus Flask, Gunicorn, Pillow, NumPy and PyYAML). The .dockerignore whitelists only app.py, params.yaml, requirements-app.txt and the model file, so the build context is a few megabytes and never includes the dataset or virtual environment. The container runs as a non-root user, exposes port 8080 and defines a HEALTHCHECK.

```
FROM python:3.11-slim

ENV PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1 \
    TF_CPP_MIN_LOG_LEVEL=2 \
    PORT=8080

WORKDIR /app

COPY requirements-app.txt .
RUN pip install --no-cache-dir -r requirements-app.txt

COPY params.yaml app.py ./
# The model is produced by `dvc repro` and fetched with `dvc pull` before the build.
COPY artifacts/model/mobilenetv2_intel_classifier.keras artifacts/model/

RUN useradd --create-home appuser
USER appuser

EXPOSE 8080
HEALTHCHECK --interval=30s --timeout=5s --start-period=60s \
    CMD python -c "import urllib.request; urllib.request.urlopen('http://localhost:8080/health')"

CMD ["gunicorn", "--bind", "0.0.0.0:8080", "--workers", "1", "--threads", "4", "--timeout", "120", "app:app"]
```

```
dvc pull model_training
docker build -t image-classifier:local .
docker run -d -p 8080:8080 image-classifier:local
curl http://localhost:8080/health
curl -F file=@samples/forest.jpg http://localhost:8080/predict
```

Docker build

```
PS> docker build -t image-classifier:local .
#1 [internal] load build definition from Dockerfile
#1 DONE 0.1s
#2 [internal] load metadata for docker.io/library/python:3.11-slim
#2 DONE 2.2s
#3 [internal] load .dockerignore
#3 DONE 0.1s
#4 [1/7] FROM
#4 DONE 0.1s
#5 [internal] load build context
#5 DONE 0.9s
#6 [2/7] WORKDIR /app
#7 [3/7] COPY requirements-app.txt .
#8 [4/7] RUN pip install --no-cache-dir -r requirements-app.txt
#9 [5/7] COPY params.yaml app.py ./
#10 [6/7] COPY artifacts/model/mobilenetv2_intel_classifier.keras artifacts/model/
#10 DONE 0.3s
#11 [7/7] RUN useradd --create-home appuser
#11 DONE 1.3s
#12 naming to docker.io/library/image-classifier:local done
#12 DONE 2.0s

PS> docker images image-classifier
REPOSITORY         TAG       IMAGE ID       SIZE
image-classifier   local     107297ceb3f0   2.2GB
```

Figure 10: Docker build of the application image (2.2 GB, mostly TensorFlow)

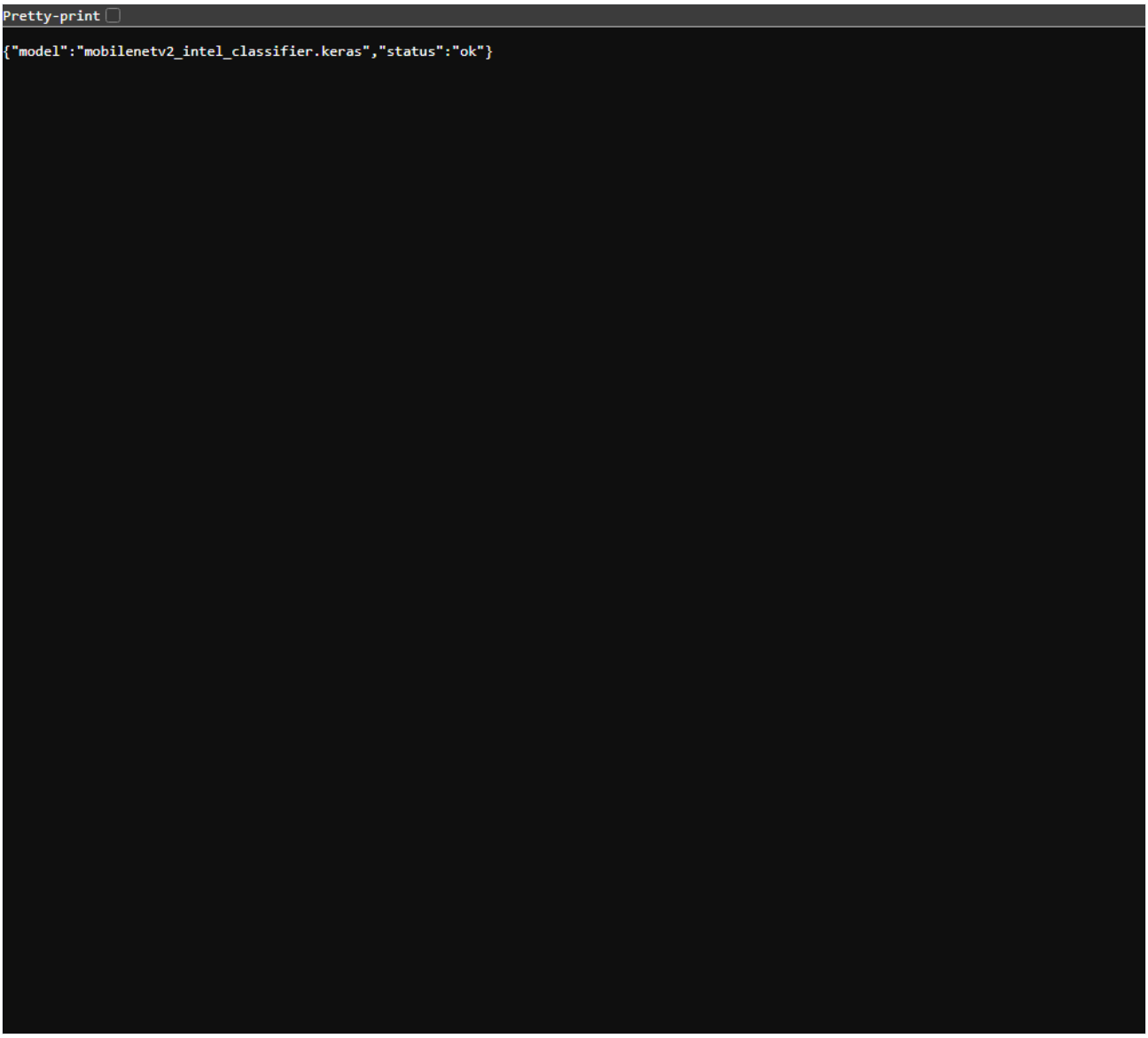


Figure 11: Health endpoint of the local container

8. Step 5 - AWS EC2 Instance

Setting	Value
Name / region	mlops-image-classifier, ap-south-1 (Mumbai)
AMI / type	Ubuntu Server LTS, t2.micro (free-tier eligible, 1 vCPU, 1 GB RAM)
Storage	20 GiB gp3 (within the 30 GB free tier)
Key pair	mlops-key (RSA, .pem) used by GitHub Actions for SSH
Security group	Inbound SSH 22 and HTTP 80 from 0.0.0.0/0
Public IPv4	15.252.12.60

The instance was bootstrapped with the following user-data script, which installs Docker and adds a 2 GB swap file so that TensorFlow fits comfortably in 1 GB of RAM:

```
#!/bin/bash
fallocate -l 2G /swapfile && chmod 600 /swapfile && mkswap /swapfile && swapon /swapfile
echo '/swapfile none swap sw 0 0' >> /etc/fstab
apt-get update -y && apt-get install -y docker.io
systemctl enable --now docker
usermod -aG docker ubuntu
```

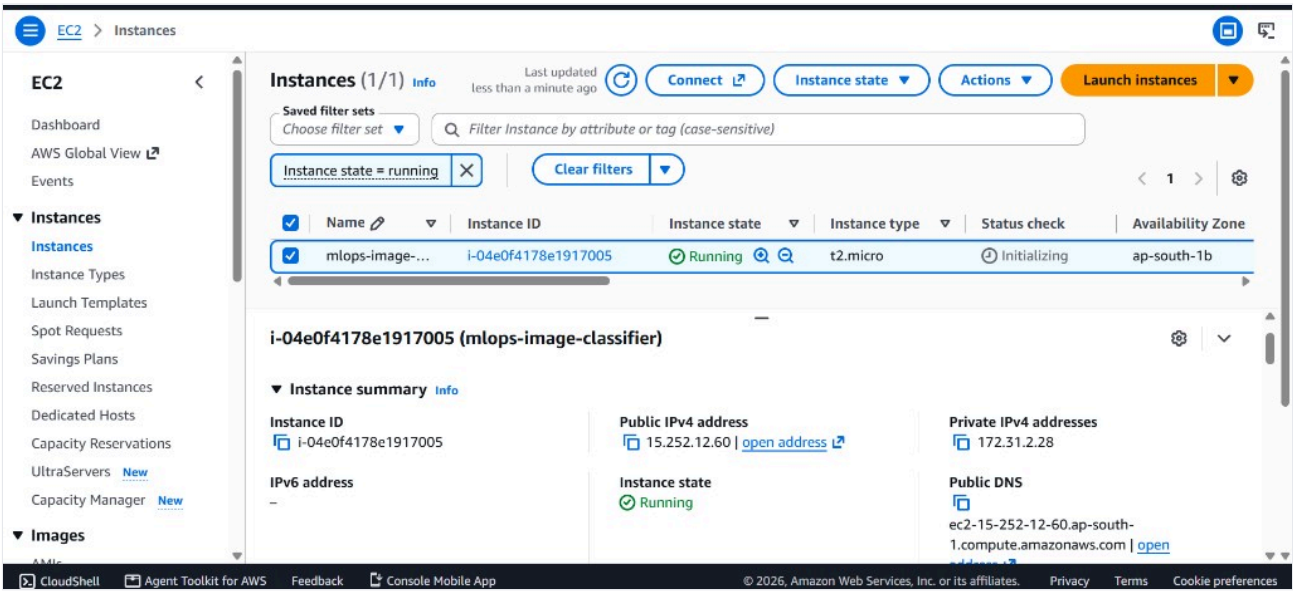


Figure 12: EC2 t2.micro instance running in ap-south-1 with public IP 15.252.12.60

9. Step 6 - GitHub Repository and Secrets

The code was pushed to the public repository https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144. The workflow uses four encrypted repository secrets, so no credential is stored in the code:

Secret	Used for
AWS_ACCESS_KEY_ID / AWS_SECRET_ACCESS_KEY	dvc pull of the model from the S3 remote
EC2_HOST	Public IP of the EC2 instance
EC2_SSH_KEY	Private key of the EC2 key pair for SSH deployment

Publishing to GitHub Container Registry (ghcr.io) uses the built-in GITHUB_TOKEN, so no extra registry account is needed.

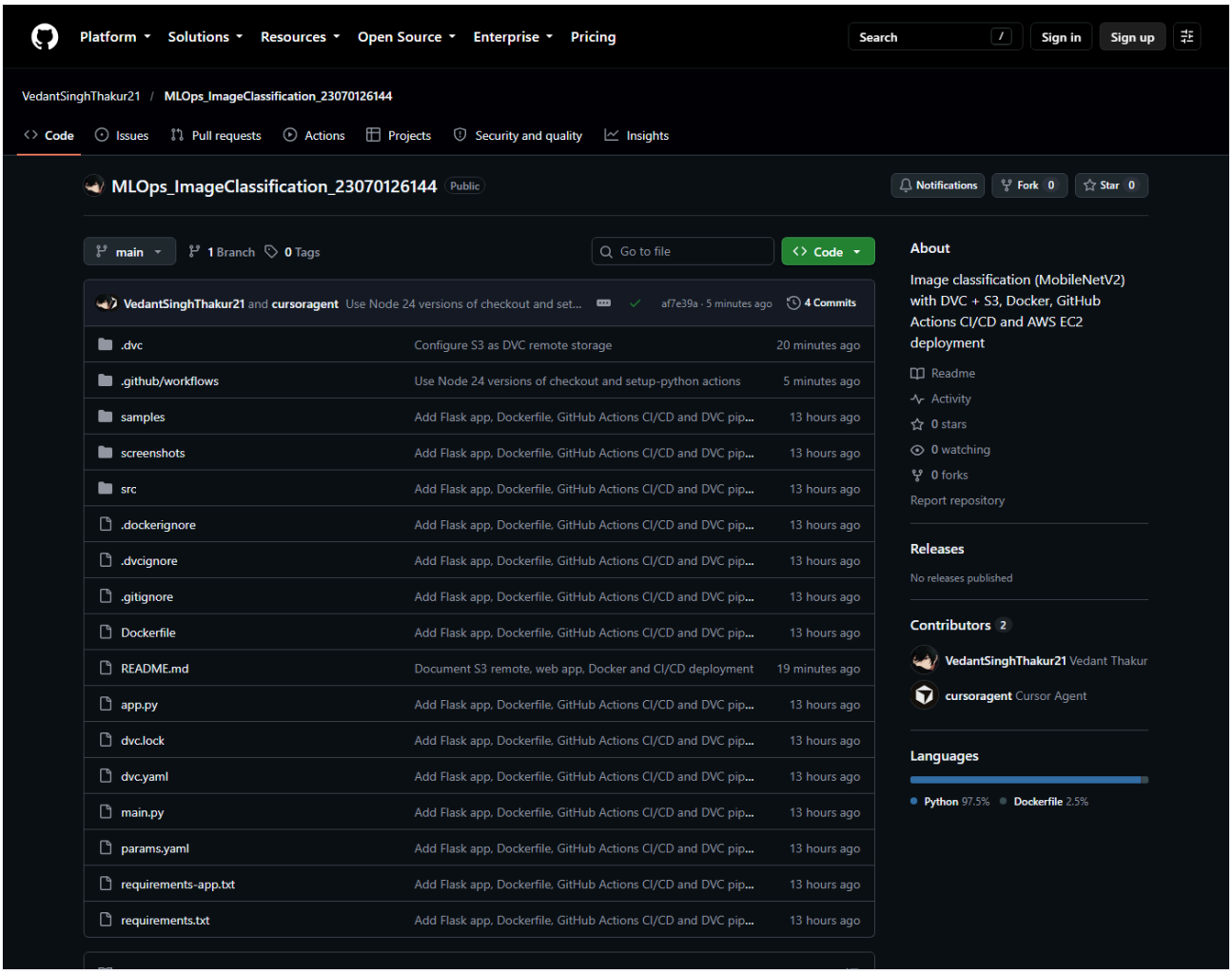


Figure 13: Public GitHub repository

10. Step 7 - GitHub Actions CI/CD (ci-cd.yaml)

The workflow runs on every push and pull request to main, and can also be started manually. It has two jobs:

1. build-test: checks out the code, installs DVC, pulls the trained model and metrics from S3 (dvc pull), prints dvc dag and dvc metrics show, builds the Docker image, starts it and smoke-tests it by asserting that samples/forest.jpg is classified as forest, then pushes the image to ghcr.io tagged with the commit SHA and latest.
2. deploy (only on main): connects to EC2 over SSH, logs in to ghcr.io with the short-lived workflow token, pulls the new image, replaces the running container (docker run -d --restart unless-stopped -p 80:8080), prunes old images and finally checks the /health endpoint on the EC2 public IP from the internet.

```yaml
name: CI/CD

on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:

permissions:
  contents: read
  packages: write

jobs:
  build-test:
    name: DVC pull, Docker build and smoke test
    runs-on: ubuntu-latest
    env:
      AWS_ACCESS_KEY_ID: ${{ secrets.AWS_ACCESS_KEY_ID }}
      AWS_SECRET_ACCESS_KEY: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
    steps:
      - uses: actions/checkout@v5

      - uses: actions/setup-python@v6
        with:
          python-version: "3.11"

      - name: Install DVC
        run: pip install "dvc[s3]"

      - name: Pull trained model and metrics from S3 (DVC remote)
        run: dvc pull model_training model_evaluation

      - name: Show DVC pipeline and metrics
        run: |
          dvc dag
          dvc metrics show

      - name: Build Docker image
        run: |
          echo "IMAGE=ghcr.io/${GITHUB_REPOSITORY,,}" >> "$GITHUB_ENV"
          docker build -t "ghcr.io/${GITHUB_REPOSITORY,,}:${GITHUB_SHA}" -t "ghcr.io/${GITHUB_REPOSITORY,,}:latest"
.

      - name: Smoke test container
        run: |
          docker run -d --name app -p 8080:8080 "$IMAGE:latest"
          for i in $(seq 1 30); do curl -fs localhost:8080/health && break; sleep 2; done
          curl -fs -F file=@samples/forest.jpg localhost:8080/predict | tee prediction.json
          python -c "import json; r = json.load(open('prediction.json')); assert r['label'] == 'forest', r"

      - name: Container logs
        if: always()
        run: docker logs app || true

      - name: Push image to GitHub Container Registry
        if: github.event_name != 'pull_request'
        run: |
          echo "${{ secrets.GITHUB_TOKEN }}" | docker login ghcr.io -u "${{ github.actor }}" --password-stdin
          docker push "$IMAGE:${GITHUB_SHA}"
          docker push "$IMAGE:latest"

  deploy:
    name: Deploy to AWS EC2
    needs: build-test
    if: github.event_name != 'pull_request'
    runs-on: ubuntu-latest
    env:
      EC2_HOST: ${{ secrets.EC2_HOST }}
      EC2_USER: ubuntu
    steps:
```

```
      - name: Deploy container over SSH
        run: |
          IMAGE="ghcr.io/${GITHUB_REPOSITORY,,}:${GITHUB_SHA}"
          install -m 600 /dev/null key.pem
          echo "${{ secrets.EC2_SSH_KEY }}" > key.pem
          ssh -i key.pem -o StrictHostKeyChecking=accept-new "$EC2_USER@$EC2_HOST" "
            echo '${{ secrets.GITHUB_TOKEN }}' | docker login ghcr.io -u '${{ github.actor }}' --password-stdin &&
            docker pull $IMAGE &&
            docker logout ghcr.io &&
            (docker rm -f image-classifier || true) &&
            docker run -d --name image-classifier --restart unless-stopped -p 80:8080 $IMAGE &&
            docker image prune -af
          "

      - name: Verify public endpoint
        run: curl -fs --retry 20 --retry-delay 6 --retry-all-errors "http://$EC2_HOST/health"
```

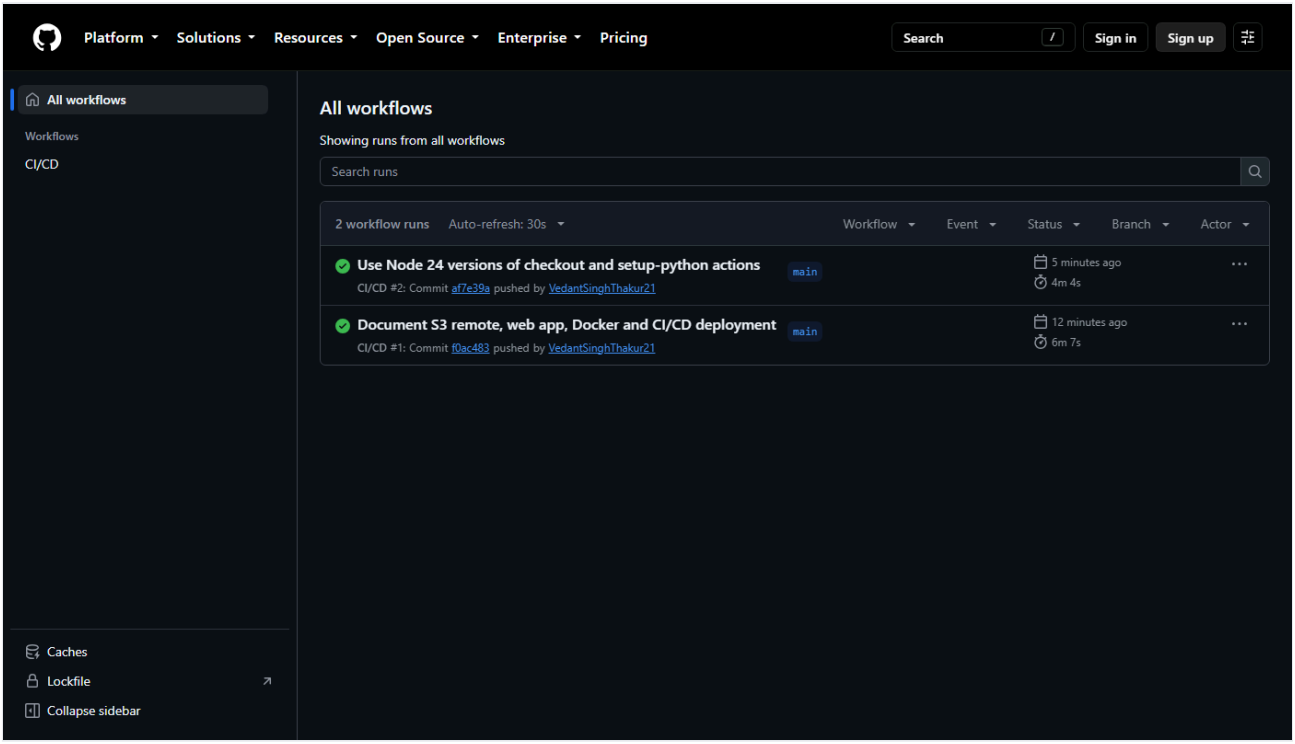


Figure 14: Successful CI/CD workflow runs

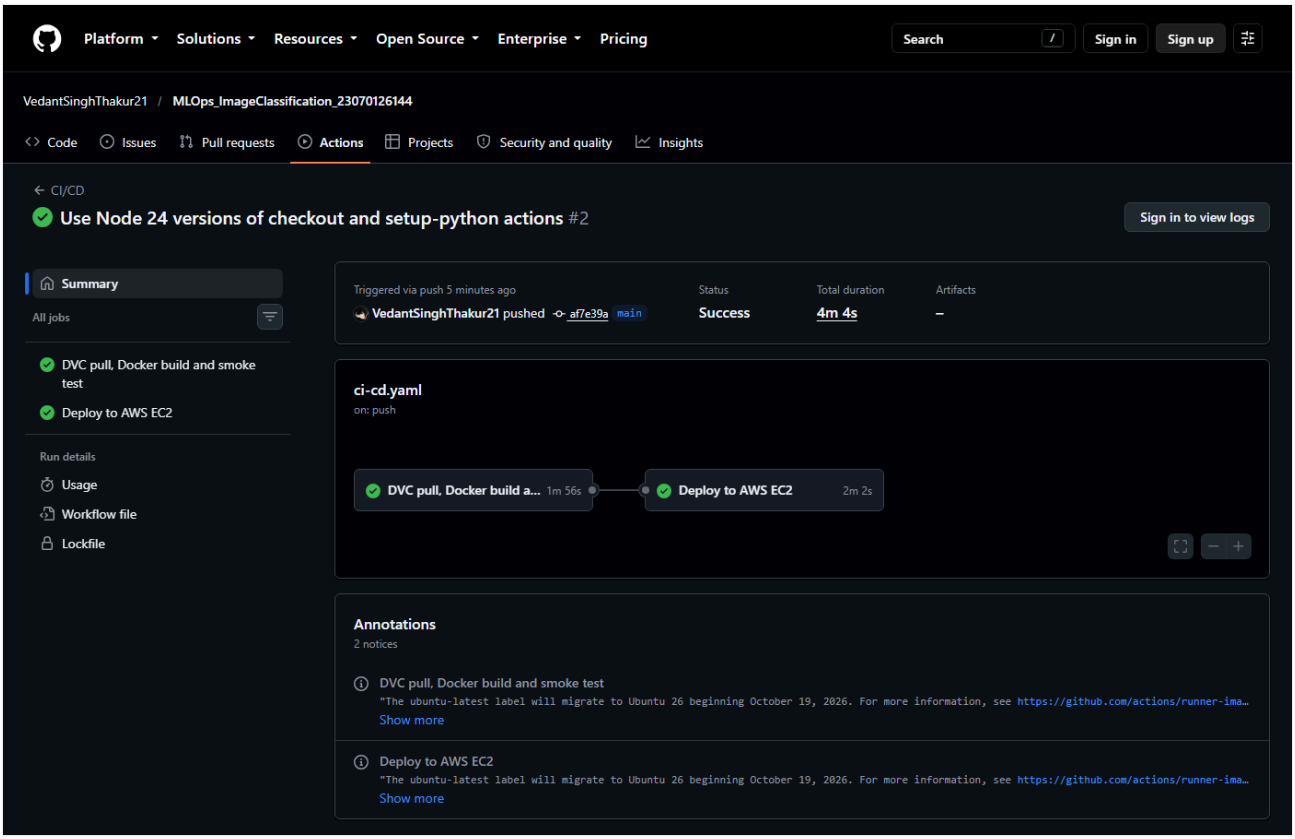


Figure 15: Run #2: build-test (1m 56s) followed by deploy to AWS EC2 (2m 2s)



Figure 16: Job logs: dvc pull from S3, DVC DAG and metrics, Docker build, smoke test, GHCR push and SSH deploy

11. Step 8 - Deployment Verification on EC2

The application is publicly available at http://15.252.12.60/. The container started by the pipeline is reported as healthy by Docker, and predictions made over the internet match the local results.

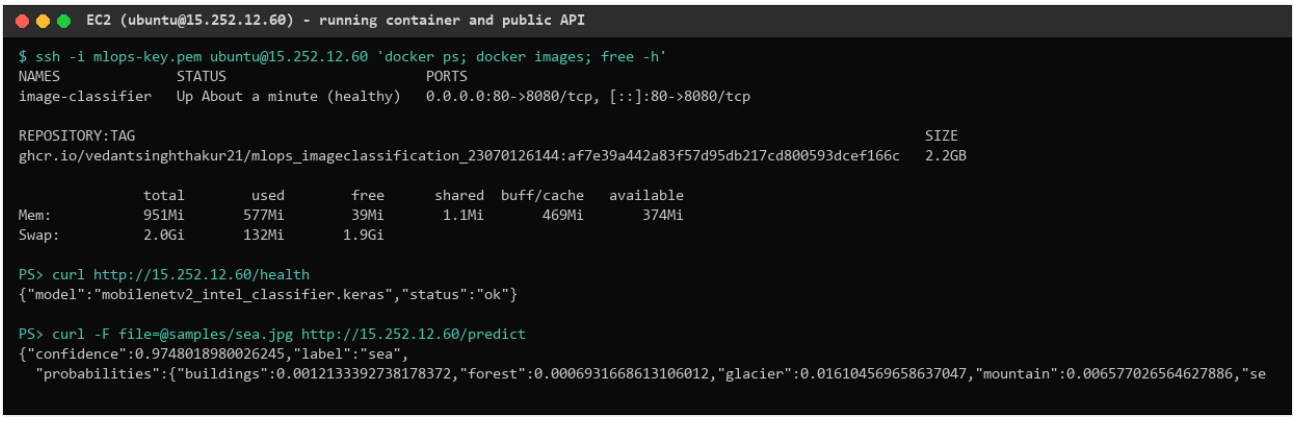


Figure 17: Container running on EC2 and the public health and prediction API

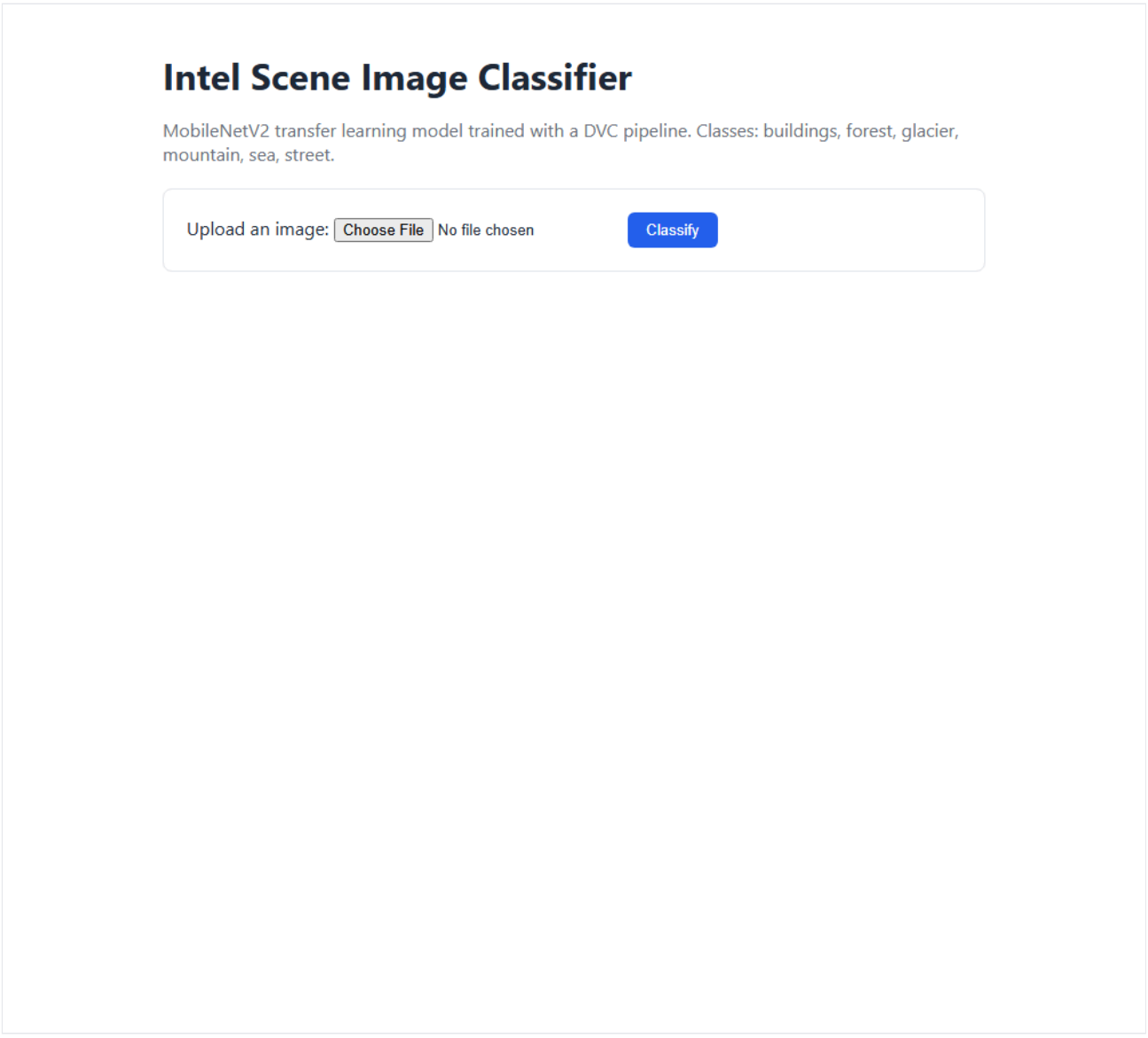


Figure 18: Application home page served from the EC2 public IP (http://15.252.12.60/)

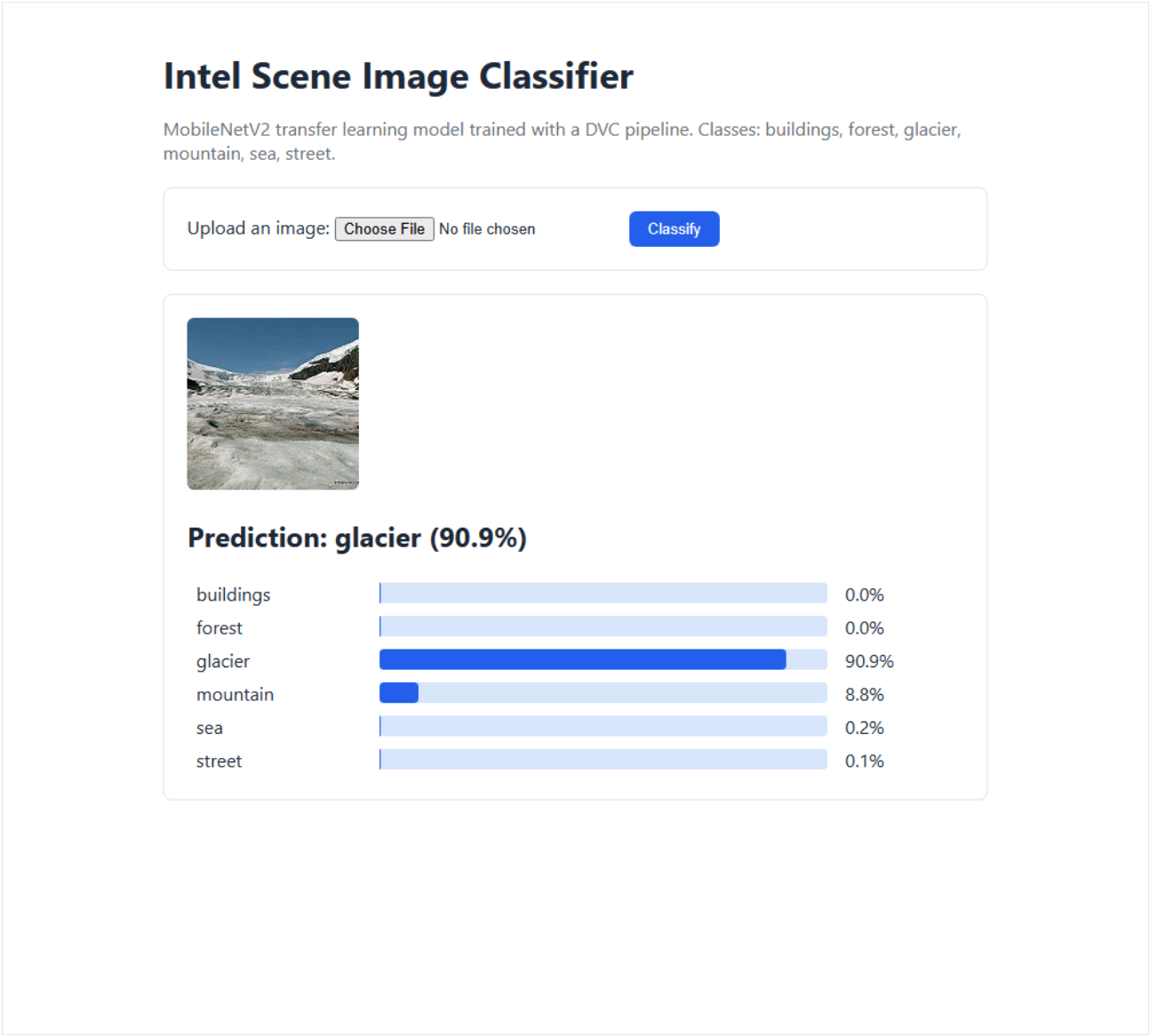


Figure 19: Prediction for a glacier image on the deployed application

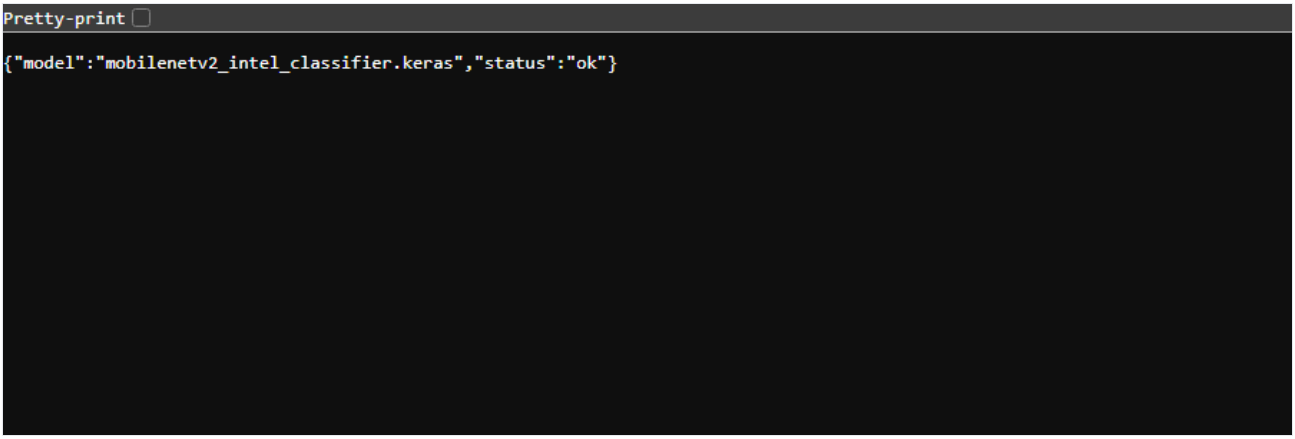


Figure 20: Health endpoint on the EC2 public IP

12. Results

Metric (test set, 300 images)	Value
Accuracy	90.67%

Precision (weighted)	90.75%
Recall (weighted)	90.67%
F1 score (weighted)	90.66%
Docker image size	2.2 GB
CI/CD duration (run #2)	4 min 4 s (build-test 1m 56s, deploy 2m 2s)

13. Challenges and Solutions

Challenge	Solution
The data_collection stage used the Kaggle CLI, which requires a Kaggle account and API token on every machine.	Switched to Kaggle's public download endpoint using Python's urllib, removing the kaggle dependency, so dvc repro works on a fresh clone.
dvc repro failed with ModuleNotFoundError: matplotlib because DVC launched the system Python instead of the virtual environment.	Activated the virtual environment (put .venv/Scripts first on PATH) before running dvc repro.
The project was initialised with DVC no_scm mode and dvc.lock was in .gitignore, so CI could not know which artifact versions to pull.	Initialised Git, removed no_scm and committed dvc.lock together with dvc.yaml and params.yaml.
The local AWS CLI credentials were invalid (InvalidClientTokenId).	Created a dedicated IAM user with only S3 access and a new access key, configured with aws configure.
TensorFlow images are large and a t2.micro has only 1 GB of RAM.	Used tensorflow-cpu with a separate slim requirements-app.txt, a whitelist .dockerignore, a single Gunicorn worker and a 2 GB swap file. Images are built on GitHub runners, not on the EC2 instance.
A model saved with one Keras version may not load with another.	Pinned tensorflow-cpu 2.21.0 and keras 3.15.1 in the container to match the training environment.
Staying strictly in the AWS free tier.	t2.micro in standard CPU-credit mode, a single instance, 20 GB disk, no Elastic IP, and only 9 small objects pushed to S3 (well below the 2,000 free PUT requests).
Container image names must be lowercase, but the GitHub repository name contains capitals.	Lowercased the name in the workflow with the bash expansion ${GITHUB_REPOSITORY,,}.
Keeping credentials out of the code.	Stored AWS keys, the EC2 host and the SSH key as encrypted GitHub secrets. GitHub masks them in logs, and the ghcr.io login on EC2 uses the short-lived workflow token followed by docker logout.
GitHub warned that Node.js 20 based actions are deprecated.	Upgraded to actions/checkout@v5 and actions/setup-python@v6.

14. Conclusion

The project demonstrates a complete MLOps workflow: a reproducible DVC pipeline trains a MobileNetV2 scene classifier with 90.7% test accuracy, its artifacts are versioned in AWS S3, and every push to GitHub automatically pulls the model, builds and tests a Docker image, publishes it and redeploys it to an AWS EC2 instance, where it is publicly reachable.

Appendix A - dvc.yaml

```
stages:
  data_collection:
    cmd: python -m src.data_collection
    deps:
      - src/data_collection.py
      - src/utils.py
      - params.yaml
    params:
      - dataset_url
      - dataset_path
      - artifacts_path
      - seed
    outs:
      - data/raw

  data_processing:
    cmd: python -m src.data_processing
    deps:
      - src/data_processing.py
      - src/utils.py
      - params.yaml
      - data/raw
    params:
      - processed_path
      - image_size
      - batch_size
      - validation_split
      - test_split
      - rotation_range
      - zoom_range
      - horizontal_flip
      - brightness_range
      - seed
      - use_augmentation
    outs:
      - data/processed
      - artifacts/processed

  feature_selection:
    cmd: python -m src.feature_selection
    deps:
      - src/feature_selection.py
      - src/utils.py
      - params.yaml
      - data/processed
    params:
      - image_size
      - model_name
      - freeze_base_model
      - fine_tune_at
    outs:
      - artifacts/features

  model_training:
    cmd: python -m src.model_training
    deps:
      - src/model_training.py
      - src/utils.py
      - params.yaml
      - data/processed
      - artifacts/features
    params:
      - batch_size
      - epochs
      - learning_rate
      - optimizer
      - dropout_rate
```

```
      - dense_units
      - loss_function
      - metrics
      - model_name
      - seed
    outs:
      - artifacts/model

  model_evaluation:
    cmd: python -m src.model_evaluation
    deps:
      - src/model_evaluation.py
      - src/utils.py
      - params.yaml
      - data/processed
      - artifacts/model
    params:
      - batch_size
      - metrics
      - seed
    metrics:
      - artifacts/metrics/metrics.json
    outs:
      - artifacts/metrics/confusion_matrix.png
```

GitHub Repository and Submission Links

GitHub repository (complete source code, public):
https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144

Deployed application (AWS EC2 public IP): http://15.252.12.60/

Item	Link / location
GitHub repository (public)	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144
Deployed application (EC2 public IP)	http://15.252.12.60/
Health check endpoint	http://15.252.12.60/health
Prediction API (POST, form field 'file')	http://15.252.12.60/predict
dvc.yaml	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/blob/main/dvc.yaml
ci-cd.yaml	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/blob/main/.github/workflows/ci-cd.yaml
Dockerfile	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/blob/main/Dockerfile
Flask application (app.py)	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/blob/main/app.py
GitHub Actions CI/CD runs	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/actions

Successful CI/CD run #2	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/actions/runs/39301239608
Screenshots	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/tree/main/screenshots
Project report (PDF / Word)	https://github.com/VedantSinghThakur21/MLOps_ImageClassification_23070126144/tree/main/report
DVC remote (AWS S3, ap-south-1)	s3://mlops-imgcls-23070126144/dvcstore
Docker image (GitHub Container Registry, private)	ghcr.io/vedantsinghthakur21/mlops_imageclassification_23070126144